

# HookSuite — Manual Técnico P4 Chrome DevTools

---

**Proyecto:** HookSuite

**Rol:** P4 — Chrome DevTools / Captura de red

**Stack:** Python + Chrome DevTools Protocol + websockets + httpx

**Última actualización:** Abril 2026

---

## Antes de empezar

Este manual es tu guía completa de trabajo en el proyecto HookSuite. Está diseñado para que puedas trabajar de forma autónoma desde cualquier lugar y en cualquier momento.

Cuando un paso requiere coordinación con otro rol:

 **CONTACTO CON P2 Backend** — Lo que necesitas pedirle y cuándo.

Cuando la IA puede ayudarte:

 **PROMPT DE IA** — Copia este prompt en Claude Code o Claude.ai.

**Sobre tu rol:** P4 produce los datos en bruto que alimentan al módulo de IA de P5 y al panel de red de P1. Tu trabajo es independiente durante las semanas 1 y 2. Además tienes asignada la elaboración del road map de la Práctica 2, que vale el 10% de la nota.

---

## Herramientas necesarias

- **Python 3.11+** — `python --version`
  - **pip** — `pip --version`
  - **Git** — `git --version`
  - **Google Chrome** — La versión más reciente
  - **Docker Desktop** — Para DVWA. `docker --version`
  - **Claude Code** — Cliente terminal de Anthropic
  - **Visual Studio Code** — O el editor que prefieras
- 

## BLOQUE 01 — Setup de Chrome DevTools Protocol

---

### Paso 1 — Clonar el repositorio y crear tu rama

#### 1.1 Abre la terminal:

```
git clone https://github.com/[URL-DEL-REPO]/hooksuite.git
cd hooksuite
git checkout -b feature/devtools
cd devtools
```

---

## Paso 2 — Configurar el entorno Python

### 2.1 Crea el entorno virtual y actívalo:

```
python -m venv venv
source venv/bin/activate # Mac/Linux
# venv\Scripts\activate # Windows
```

### 2.2 Crea requirements.txt:

```
websockets==12.0
httpx==0.27.0
python-dotenv==1.0.1
```

### 2.3 Instala las dependencias:

```
pip install -r requirements.txt
```

### 2.4 Crea el archivo .env:

```
BACKEND_URL=http://localhost:8000
CHROME_DEBUG_PORT=9222
DVWA_URL=http://localhost:8888
SESSION_TOKEN=devtools_session
```

---

## Paso 3 — Levantar DVWA

```
docker run -d --name dvwa -p 8888:80 vulnerables/web-dvwa
```

Espera 10 segundos, abre <http://localhost:8888>, haz login con [admin/password](#). Si aparece la pantalla de setup haz clic en "Create / Reset Database". Cambia el nivel de seguridad a **Low** desde DVWA Security.

---

## Paso 4 — Crear la estructura de carpetas

```
mkdir -p core analyzers results utils
touch core/__init__.py analyzers/__init__.py utils/__init__.py
```

Estructura final:

```
devtools/
├── core/
│   ├── __init__.py
│   ├── cdp_client.py      ← Cliente CDP via WebSocket
│   ├── chrome_launcher.py ← Lanzador de Chrome con debugging
│   └── packet_builder.py  ← Constructor del objeto paquete
├── analyzers/
│   ├── __init__.py
│   ├── network_analyzer.py ← Análisis de tráfico y patrones
│   └── console_analyzer.py ← Análisis de logs de consola
├── results/
├── utils/
│   ├── __init__.py
│   └── reporter.py        ← Envío de paquetes al backend P2
├── main.py
├── requirements.txt
└── .env
```

## Paso 5 — Crear el lanzador de Chrome

El Chrome DevTools Protocol (CDP) permite acceder a los datos de red, consola e inspector del navegador. Para usarlo, Chrome debe arrancarse con el flag `--remote-debugging-port`.

### 5.1 Crea `core/chrome_launcher.py`:

```
import subprocess
import sys
import os
import time
import httpx
from dotenv import load_dotenv

load_dotenv()
DEBUG_PORT = int(os.getenv("CHROME_DEBUG_PORT", 9222))

def get_chrome_path() -> str:
    if sys.platform == "darwin":
        return "/Applications/Google Chrome.app/Contents/MacOS/Google Chrome"
    elif sys.platform == "win32":
        for path in [
```

```

        r"C:\Program Files\Google\Chrome\Application\chrome.exe",
        r"C:\Program Files (x86)\Google\Chrome\Application\chrome.exe",
    ]:
        if os.path.exists(path):
            return path
    else:
        for path in ["/usr/bin/google-chrome", "/usr/bin/chromium-browser"]:
            if os.path.exists(path):
                return path
    return "google-chrome"

def launch_chrome_with_debugging(url: str = "about:blank") -> subprocess.Popen:
    chrome_path = get_chrome_path()
    args = [
        chrome_path,
        f"--remote-debugging-port={DEBUG_PORT}",
        "--no-first-run",
        "--no-default-browser-check",
        "--disable-extensions",
        url,
    ]
    process = subprocess.Popen(args, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
    print(f"✓ Chrome lanzado con debugging en puerto {DEBUG_PORT} (PID:
{process.pid})")
    time.sleep(2)
    return process

async def verify_cdp_connection() -> bool:
    try:
        async with httpx.AsyncClient(timeout=5) as client:
            response = await client.get(f"http://localhost:{DEBUG_PORT}/json")
            tabs = response.json()
            if tabs:
                print(f"✓ CDP activo. {len(tabs)} pestaña(s) disponible(s)")
                for tab in tabs:
                    print(f" → {tab.get('title', 'Sin título')} – {tab.get('url',
''}})")
                return True
            except Exception as e:
                print(f"X CDP no disponible en puerto {DEBUG_PORT}: {e}")
    return False

```

## 5.2 Prueba el lanzador creando `verify_cdp.py` temporalmente:

```

import asyncio
from core.chrome_launcher import launch_chrome_with_debugging,
verify_cdp_connection

async def main():
    process = launch_chrome_with_debugging("http://localhost:8888")
    await asyncio.sleep(2)

```

```
connected = await verify_cdp_connection()
if connected:
    print("✓ Chrome DevTools Protocol funciona correctamente.")
    process.terminate()

asyncio.run(main())
```

```
python verify_cdp.py
```

Debes ver ✓ Chrome DevTools Protocol funciona correctamente. Elimina el script:

```
rm verify_cdp.py
```

 **PROMPT DE IA** — Si Chrome no arranca o el CDP no responde:

Estoy intentando lanzar Google Chrome con `--remote-debugging-port=9222` para HookSuite en [Mac/Windows/Linux]. Cuando ejecuto el comando recibo este error: [pega el error]. ¿Cómo lo soluciono?

## Paso 6 — Crear el cliente CDP

### 6.1 Crea `core/cdp_client.py`:

```
import asyncio
import json
import websockets
import httpx
from typing import Callable, Dict
from dotenv import load_dotenv
import os

load_dotenv()
DEBUG_PORT = int(os.getenv("CHROME_DEBUG_PORT", 9222))

class CDPClient:
    def __init__(self):
        self.ws = None
        self.message_id = 0
        self.pending_commands = {}
        self.event_handlers: Dict[str, list] = {}
        self.connected = False

    async def connect(self, tab_url: str = None) -> bool:
        try:
```

```

    async with httpx.AsyncClient(timeout=5) as client:
        response = await client.get(f"http://localhost:{DEBUG_PORT}/json")
        tabs = response.json()

    if not tabs:
        print("X No hay pestañas disponibles en Chrome")
        return False

    target_tab = None
    if tab_url:
        for tab in tabs:
            if tab_url in tab.get("url", ""):
                target_tab = tab
                break
    if not target_tab:
        target_tab = tabs[0]

    ws_url = target_tab.get("websocketDebuggerUrl")
    if not ws_url:
        print("X La pestaña no tiene WebSocket debugger URL")
        return False

    self.ws = await websockets.connect(ws_url, max_size=100 * 1024 * 1024)
    self.connected = True
    print(f"✓ Conectado al CDP: {target_tab.get('title', 'Sin título')}")
    asyncio.create_task(self._listen())
    return True

except Exception as e:
    print(f"X Error conectando al CDP: {e}")
    return False

async def _listen(self):
    try:
        async for message in self.ws:
            data = json.loads(message)
            if "id" in data and data["id"] in self.pending_commands:
                future = self.pending_commands.pop(data["id"])
                if not future.done():
                    future.set_result(data.get("result", {}))
            if "method" in data:
                method = data["method"]
                for handler in self.event_handlers.get(method, []):
                    asyncio.create_task(handler(data.get("params", {})))
    except websockets.exceptions.ConnectionClosed:
        self.connected = False
    except Exception as e:
        print(f"X Error en listener CDP: {e}")

async def send(self, method: str, params: dict = None) -> dict:
    if not self.connected:
        return {}
    self.message_id += 1
    msg_id = self.message_id

```

```

        message = {"id": msg_id, "method": method, "params": params or {}}
        future = asyncio.get_event_loop().create_future()
        self.pending_commands[msg_id] = future
        await self.ws.send(json.dumps(message))
    try:
        return await asyncio.wait_for(future, timeout=10.0)
    except asyncio.TimeoutError:
        self.pending_commands.pop(msg_id, None)
        return {}

def on(self, event: str, handler: Callable):
    if event not in self.event_handlers:
        self.event_handlers[event] = []
    self.event_handlers[event].append(handler)

async def enable_network(self):
    await self.send("Network.enable")
    print("✓ Network domain activado")

async def enable_console(self):
    await self.send("Console.enable")
    print("✓ Console domain activado")

async def enable_page(self):
    await self.send("Page.enable")
    print("✓ Page domain activado")

async def navigate(self, url: str):
    await self.send("Page.navigate", {"url": url})
    await asyncio.sleep(2)

async def get_response_body(self, request_id: str) -> str:
    result = await self.send("Network.getResponseBody", {"requestId":
request_id})
    body = result.get("body", "")
    if result.get("base64Encoded"):
        import base64
        try:
            body = base64.b64decode(body).decode("utf-8", errors="replace")
        except Exception:
            body = "[binary content]"
    return body

async def close(self):
    if self.ws:
        await self.ws.close()
    self.connected = False
    print("✓ Conexión CDP cerrada")

```

## Paso 7 — Crear el constructor de paquetes

📖 **CONTACTO CON P2 Backend** — Antes de implementar este paso confirma con P2 el formato exacto del objeto paquete. El formato base que usamos es:

```
{
  "id": "uuid",
  "method": "GET",
  "url": "http://...",
  "status": 200,
  "size": 1024,
  "time": 145,
  "timestamp": "ISO-8601",
  "request_headers": {},
  "request_body": null,
  "response_headers": {},
  "response_body": "...",
  "suspicious": false,
  "vulnerable": false
}
```

Si P2 necesita campos adicionales, actualiza el método `build_packet()`.

### 7.1 Crea `core/packet_builder.py`:

```
import uuid
import time
from datetime import datetime
from typing import Dict, Optional

IGNORED_DOMAINS = [
    'google-analytics.com', 'googletagmanager.com', 'doubleclick.net',
    'fonts.googleapis.com', 'fonts.gstatic.com', 'ajax.googleapis.com',
    'cdnjs.cloudflare.com', 'cdn.cloudflare.com',
]

IGNORED_RESOURCE_TYPES = ['Image', 'Stylesheet', 'Font', 'Media', 'Ping']
IGNORED_EXTENSIONS = ['.css', '.woff', '.woff2', '.ttf', '.ico',
                      '.png', '.jpg', '.jpeg', '.gif', '.svg', '.webp']

class PacketBuilder:
    def __init__(self):
        self.pending_requests: Dict[str, dict] = {}
        self.completed_packets = []

    def should_filter(self, url: str, resource_type: str = "") -> bool:
        url_lower = url.lower()
        for domain in IGNORED_DOMAINS:
            if domain in url_lower:
                return True
        for ext in IGNORED_EXTENSIONS:
            if url_lower.split('?')[0].endswith(ext):
                return True
```



```

        if resource_type in IGNORED_RESOURCE_TYPES:
            return True
        return False

def on_request_sent(self, params: dict):
    request_id = params.get("requestId")
    request = params.get("request", {})
    url = request.get("url", "")
    resource_type = params.get("type", "")
    if self.should_filter(url, resource_type):
        return None
    self.pending_requests[request_id] = {
        "id": str(uuid.uuid4()),
        "request_id": request_id,
        "method": request.get("method", "GET"),
        "url": url,
        "request_headers": dict(request.get("headers", {})),
        "request_body": request.get("postData"),
        "timestamp": datetime.utcnow().isoformat(),
        "start_time": time.time(),
    }
    return request_id

def on_response_received(self, params: dict):
    request_id = params.get("requestId")
    if request_id not in self.pending_requests:
        return
    response = params.get("response", {})
    self.pending_requests[request_id].update({
        "status": response.get("status", 0),
        "response_headers": dict(response.get("headers", {})),
        "mime_type": response.get("mimeType", ""),
    })

def build_packet(self, request_id: str, response_body: str = "") ->
Optional[dict]:
    if request_id not in self.pending_requests:
        return None
    req_data = self.pending_requests.pop(request_id)
    elapsed = int((time.time() - req_data["start_time"]) * 1000)
    packet = {
        "id": req_data["id"],
        "method": req_data["method"],
        "url": req_data["url"],
        "status": req_data.get("status", 0),
        "size": len(response_body.encode("utf-8", errors="replace")),
        "time": elapsed,
        "timestamp": req_data["timestamp"],
        "request_headers": req_data["request_headers"],
        "request_body": req_data["request_body"],
        "response_headers": req_data.get("response_headers", {}),
        "response_body": response_body[:50000],
        "suspicious": False,
        "vulnerable": False,
    }

```

```

    }
    self.completed_packets.append(packet)
    return packet

def get_completed_packets(self) -> list:
    return self.completed_packets.copy()

```

## Paso 8 — Commit del bloque 01

```

git add .
git commit -m "feat(devtools): cliente CDP, constructor de paquetes y launcher de Chrome"
git push origin feature/devtools

```

## BLOQUE 02 — Captura y análisis de tráfico

### Paso 9 — Crear el analizador de red

#### 9.1 Crea `analyzers/network_analyzer.py`:

```

import asyncio
from typing import List, Callable, Dict
from core.cdp_client import CDPClient
from core.packet_builder import PacketBuilder

SECURITY_HEADERS_REQUIRED = [
    "content-security-policy", "x-frame-options",
    "strict-transport-security", "x-content-type-options", "referrer-policy",
]
SESSION_COOKIE_NAMES = ["phpsessid", "jsessionid", "session", "token", "auth", "sessionid"]
SQL_ERROR_PATTERNS = [
    "you have an error in your sql syntax", "mysql_fetch", "ora-", "pg_query",
    "sqlite_", "unclosed quotation mark", "syntax error", "database error",
]

class NetworkAnalyzer:
    def __init__(self, cdp_client: CDPClient):
        self.cdp = cdp_client
        self.packet_builder = PacketBuilder()
        self.packet_callbacks: List[Callable] = []

    def on_packet(self, callback: Callable):
        self.packet_callbacks.append(callback)

    def _check_suspicious(self, packet: dict) -> list:

```

```
reasons = []
url = packet.get("url", "").lower()
body = (packet.get("response_body") or "").lower()
req_body = (packet.get("request_body") or "").lower()
status = packet.get("status", 0)

if status >= 500:
    reasons.append(f"HTTP_ERROR_{status}")
for pattern in SQL_ERROR_PATTERNS:
    if pattern in body:
        reasons.append(f"SQL_ERROR_IN_RESPONSE")
        break
for char in ['"', "'", '<script', 'union select', '--', '1=1']:
    if char in url or char in req_body:
        reasons.append(f"INJECTION_CHAR_IN_REQUEST")
        break
if packet.get("time", 0) > 5000:
    reasons.append(f"SLOW_RESPONSE: {packet['time']}ms")
return reasons

def _check_security_headers(self, packet: dict) -> list:
    issues = []
    headers = {k.lower(): v for k, v in (packet.get("response_headers") or {}).items()}
    for h in SECURITY_HEADERS_REQUIRED:
        if h not in headers:
            issues.append(f"MISSING_HEADER: {h}")
    set_cookie = headers.get("set-cookie", "")
    if set_cookie:
        name_lower = set_cookie.lower().split("=")[0].strip()
        if any(n in name_lower for n in SESSION_COOKIE_NAMES):
            if "httponly" not in set_cookie.lower():
                issues.append("SESSION_COOKIE_MISSING_HTTPONLY")
            if "secure" not in set_cookie.lower():
                issues.append("SESSION_COOKIE_MISSING_SECURE")
    return issues

async def _handle_request_sent(self, params: dict):
    self.packet_builder.on_request_sent(params)

async def _handle_response_received(self, params: dict):
    self.packet_builder.on_response_received(params)

async def _handle_loading_finished(self, params: dict):
    request_id = params.get("requestId")
    if request_id not in self.packet_builder.pending_requests:
        return
    response_body = ""
    try:
        response_body = await self.cdp.get_response_body(request_id)
    except Exception:
        pass
    packet = self.packet_builder.build_packet(request_id, response_body)
    if not packet:
```

```

        return
    suspicious_reasons = self._check_suspicious(packet)
    security_issues = self._check_security_headers(packet)
    if suspicious_reasons:
        packet["suspicious"] = True
        packet["suspicious_reasons"] = suspicious_reasons
        print(f"  ⚠  Sospechoso: {packet['url'][:60]} → {suspicious_reasons}")
    if security_issues:
        packet["security_issues"] = security_issues
    for callback in self.packet_callbacks:
        await callback(packet)

    async def _handle_loading_failed(self, params: dict):
        request_id = params.get("requestId")
        error_text = params.get("errorText", "Unknown error")
        if request_id in self.packet_builder.pending_requests:
            packet = self.packet_builder.build_packet(request_id, f"ERROR: {error_text}")
            if packet:
                packet["network_error"] = error_text
                for callback in self.packet_callbacks:
                    await callback(packet)

    async def start(self):
        await self.cdp.enable_network()
        await self.cdp.enable_page()
        self.cdp.on("Network.requestWillBeSent", self._handle_request_sent)
        self.cdp.on("Network.responseReceived", self._handle_response_received)
        self.cdp.on("Network.loadingFinished", self._handle_loading_finished)
        self.cdp.on("Network.loadingFailed", self._handle_loading_failed)
        print("✓ Analizador de red activo")

    def get_statistics(self) -> dict:
        packets = self.packet_builder.get_completed_packets()
        suspicious = [p for p in packets if p.get("suspicious")]
        return {
            "total_packets": len(packets),
            "suspicious_packets": len(suspicious),
            "average_time_ms": sum(p.get("time", 0) for p in packets) //
max(len(packets), 1),
        }

```



**PROMPT DE IA** — Si el analizador no captura ciertos tipos de peticiones:

Tengo un módulo Python que usa Chrome DevTools Protocol para capturar tráfico de red en HookSuite. El problema es que [describe el problema: no captura peticiones AJAX / el body de respuesta llega vacío / peticiones POST sin body]. Aquí está el código: [pega el código]. ¿Cómo lo soluciono?

## Paso 10 — Crear el analizador de consola

### 10.1 Crea `analyzers/console_analyzer.py`:

```
import re
from typing import List, Dict, Callable
from core.cdp_client import CDPClient

SENSITIVE_PATTERNS = [
    (r'/var/www/', "SERVER_PATH_EXPOSED"),
    (r'/home/\w+/', "SERVER_PATH_EXPOSED"),
    (r'C:\\\\', "SERVER_PATH_EXPOSED"),
    (r'api[_-]?key\s*[:=]\s*["\']?\w-{10,}', "API_KEY_EXPOSED"),
    (r'password\s*[:=]\s*["\']?\w+', "PASSWORD_EXPOSED"),
    (r'token\s*[:=]\s*["\']?\w.-{20,}', "TOKEN_EXPOSED"),
    (r'mysql_fetch|pg_query|sqlite_exec', "DB_FUNCTION_EXPOSED"),
    (r'You have an error in your SQL', "SQL_ERROR_EXPOSED"),
    (r'ORA-\d{5}', "ORACLE_ERROR_EXPOSED"),
    (r'Stack trace:', "STACK_TRACE_EXPOSED"),
    (r'\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}', "INTERNAL_IP_EXPOSED"),
]

class ConsoleAnalyzer:
    def __init__(self, cdp_client: CDPClient):
        self.cdp = cdp_client
        self.findings: List[Dict] = []
        self.all_messages: List[Dict] = []
        self.finding_callbacks: List[Callable] = []

    def on_finding(self, callback: Callable):
        self.finding_callbacks.append(callback)

    def _analyze_message(self, text: str) -> List[str]:
        detections = []
        for pattern, detection_type in SENSITIVE_PATTERNS:
            if re.search(pattern, text, re.IGNORECASE):
                detections.append(detection_type)
        return detections

    async def _handle_console_message(self, params: dict):
        message = params.get("message", {})
        level = message.get("level", "log")
        text = message.get("text", "")
        msg_data = {"level": level, "text": text, "url": message.get("url", ""),
"line": message.get("line", 0)}
        self.all_messages.append(msg_data)
        if level in ["error", "warning"]:
            detections = self._analyze_message(text)
            if detections:
                finding = {**msg_data, "detections": detections,
                    "severity": "high" if any(d in ["API_KEY_EXPOSED",
"PASSWORD_EXPOSED", "TOKEN_EXPOSED"]
for d in detections) else
```

```

"medium"}

        self.findings.append(finding)
        print(f"  🔍 Consola [{level.upper()}]: {detections} →
{text[:80]}")

        for callback in self.finding_callbacks:
            await callback(finding)

    async def start(self):
        await self.cdp.enable_console()
        self.cdp.on("Console.messageAdded", self._handle_console_message)
        print("✓ Analizador de consola activo")

    def get_findings(self) -> List[Dict]:
        return self.findings.copy()

    def get_summary(self) -> dict:
        return {
            "total_messages": len(self.all_messages),
            "errors": len([m for m in self.all_messages if m.get("level") ==
"error"]),
            "sensitive_findings": len(self.findings),
        }

```

## Paso 11 — Crear el reporter para P2

📖 **CONTACTO CON P2 Backend** — Cuando P2 tenga listo el endpoint `POST /api/network/packet/{session_token}`, activa el envío en el reporter cambiando `self.backend_available = True` manualmente o ejecutando `check_backend()`. La URL completa será `http://localhost:8000/api/network/packet/{session_token}`.

### 11.1 Crea `utils/reporter.py`:

```

import httpx
import json
import os
from typing import Dict
from dotenv import load_dotenv
from datetime import datetime

load_dotenv()
BACKEND_URL = os.getenv("BACKEND_URL", "http://localhost:8000")
SESSION_TOKEN = os.getenv("SESSION_TOKEN", "devtools_session")

class PacketReporter:
    def __init__(self, session_token: str = None):
        self.session_token = session_token or SESSION_TOKEN
        self.backend_available = False
        self.sent_count = 0
        self.failed_count = 0
        self.local_buffer = []

```

```
self.local_log_path = "results/captured_packets.json"
os.makedirs("results", exist_ok=True)

async def check_backend(self) -> bool:
    try:
        async with httpx.AsyncClient(timeout=5) as client:
            response = await client.get(f"{BACKEND_URL}/health")
            self.backend_available = response.status_code == 200
            if self.backend_available:
                print(f"✓ Backend disponible en {BACKEND_URL}")
            return self.backend_available
    except Exception:
        print(f"⚠ Backend no disponible. Guardando en local.")
        self.backend_available = False
        return False

async def send_packet(self, packet: Dict) -> bool:
    self.local_buffer.append(packet)
    if len(self.local_buffer) % 10 == 0:
        self._flush_local()
    if not self.backend_available:
        return False
    try:
        async with httpx.AsyncClient(timeout=10) as client:
            response = await client.post(
                f"{BACKEND_URL}/api/network/packet/{self.session_token}",
                json=packet,
            )
            if response.status_code == 200:
                self.sent_count += 1
                return True
            self.failed_count += 1
            return False
    except Exception:
        self.failed_count += 1
        return False

async def send_console_finding(self, finding: Dict) -> bool:
    if not self.backend_available:
        return False
    try:
        async with httpx.AsyncClient(timeout=10) as client:
            response = await client.post(
                f"{BACKEND_URL}/api/network/console_finding/{self.session_token}",
                json=finding,
            )
            return response.status_code == 200
    except Exception:
        return False

def _flush_local(self):
    try:
        with open(self.local_log_path, 'w') as f:
```

```

        json.dump({
            "session_token": self.session_token,
            "timestamp": datetime.utcnow().isoformat(),
            "total": len(self.local_buffer),
            "packets": self.local_buffer,
        }, f, indent=2)
    except Exception as e:
        print(f"X Error guardando log local: {e}")

    def flush(self):
        self._flush_local()
        print(f"✓ {len(self.local_buffer)} paquetes en {self.local_log_path}")
        print(f" → Enviados al backend: {self.sent_count} | Fallidos: {self.failed_count}")

```

## Paso 12 — Commit del bloque 02

```

git add .
git commit -m "feat(devtools): analizadores de red y consola con detección de patrones"
git push origin feature/devtools

```

## BLOQUE 03 — Punto de entrada principal

### Paso 13 — Crear el main.py

#### 13.1 Crea `main.py`:

```

import asyncio
import os
from dotenv import load_dotenv

from core.chrome_launcher import launch_chrome_with_debugging,
verify_cdp_connection
from core.cdp_client import CDPClient
from analyzers.network_analyzer import NetworkAnalyzer
from analyzers.console_analyzer import ConsoleAnalyzer
from utils.reporter import PacketReporter

load_dotenv()

TARGET_URL = os.getenv("DVWA_URL", "http://localhost:8888")
SESSION_TOKEN = os.getenv("SESSION_TOKEN", "devtools_session")

async def run_capture(target_url: str, duration_seconds: int = 60, session_token:
str = None):

```



```

print(f"\n{'='*60}")
print(f"  HookSuite – Captura Chrome DevTools")
print(f"  Objetivo: {target_url} | Duración: {duration_seconds}s")
print(f"{'='*60}\n")

reporter = PacketReporter(session_token or SESSION_TOKEN)
await reporter.check_backend()

print("Lanzando Chrome con debugging activo...")
chrome_process = launch_chrome_with_debugging(target_url)
await asyncio.sleep(3)

connected = await verify_cdp_connection()
if not connected:
    print("X No se pudo conectar al CDP.")
    chrome_process.terminate()
    return

cdp = CDPClient()
connected = await cdp.connect(target_url)
if not connected:
    print("X No se pudo establecer la conexión WebSocket con CDP.")
    chrome_process.terminate()
    return

network_analyzer = NetworkAnalyzer(cdp)
console_analyzer = ConsoleAnalyzer(cdp)

async def on_packet(packet):
    flag = "⚠️ " if packet.get("suspicious") else " "
    print(f"  {flag}[{packet['method']}] {packet['url'][:70]} → {packet['status']} ({packet['time']}ms)")
    await reporter.send_packet(packet)

async def on_console_finding(finding):
    print(f"  🔍 CONSOLA [{finding['level'].upper()}]: {finding['detections']}")
    await reporter.send_console_finding(finding)

network_analyzer.on_packet(on_packet)
console_analyzer.on_finding(on_console_finding)

await network_analyzer.start()
await console_analyzer.start()

print(f"\n✓ Captura activa. Navega en Chrome para capturar tráfico.")
print(f"  Duración máxima: {duration_seconds}s | Presiona Ctrl+C para detener.\n")

try:
    await asyncio.sleep(duration_seconds)
except asyncio.CancelledError:
    pass

```

```

net_stats = network_analyzer.get_statistics()
console_summary = console_analyzer.get_summary()

print(f"\n{'='*60}")
print(f"  CAPTURA COMPLETADA")
print(f"  Paquetes: {net_stats['total_packets']} | Sospechosos:
{net_stats['suspicious_packets']}")
print(f"  Tiempo medio: {net_stats['average_time_ms']}ms")
print(f"  Consola – Mensajes: {console_summary['total_messages']} | Hallazgos:
{console_summary['sensitive_findings']}")
print(f"{'='*60}\n")

reporter.flush()
await cdp.close()
chrome_process.terminate()

if __name__ == "__main__":
    asyncio.run(run_capture(TARGET_URL, duration_seconds=60))

```

**13.2** Ejecuta la captura y navega por DVWA durante 30 segundos:

```
python main.py
```

Visita <http://localhost:8888/vulnerabilities/sqli/?id='&Submit=Submit> para provocar un error SQL y ver cómo se marca como sospechoso.

**13.3** Guarda la salida como evidencia:

```
python main.py 2>&1 | tee results/capture_output.txt
```

**13.4** Commit:

```

git add .
git commit -m "feat(devtools): punto de entrada principal con captura completa"
git push origin feature/devtools

```

## BLOQUE 04 — Integración con P2 y P5

### Paso 14 — Verificar integración con P2

Cuando P2 confirme que el backend está corriendo:

🟡 **CONTACTO CON P2 Backend** — Pide a P2 que confirme:

1. El servidor FastAPI corre en `http://localhost:8000`
2. El endpoint `POST /api/network/packet/{session_token}` está disponible
3. Si el formato del paquete necesita algún ajuste

#### 14.1 Crea `test_integration.py` temporalmente:

```
import asyncio
from utils.reporter import PacketReporter
from datetime import datetime

async def test():
    reporter = PacketReporter("test_integration")
    available = await reporter.check_backend()
    if available:
        test_packet = {
            "id": "test-123",
            "method": "GET",
            "url": "http://dvwa.local/test",
            "status": 200,
            "size": 1024,
            "time": 150,
            "timestamp": datetime.utcnow().isoformat(),
            "request_headers": {"Host": "dvwa.local"},
            "request_body": None,
            "response_headers": {"Content-Type": "text/html"},
            "response_body": "Test response body",
            "suspicious": False,
            "vulnerable": False,
        }
        sent = await reporter.send_packet(test_packet)
        print(f"✓ Paquete de prueba enviado: {sent}")
    else:
        print("⚠ Backend no disponible todavía. Inténtalo cuando P2 confirme que está listo.")

asyncio.run(test())
```

```
python test_integration.py
```

Cuando funcione, elimina el script:

```
rm test_integration.py
```

---

## Paso 15 — Preparar datos para P5

📌 **CONTACTO CON P5 IA** — Comparte con P5 el archivo `results/captured_packets.json` después de una captura. Es el formato exacto que consumirá. Señala especialmente:

- Los paquetes con `suspicious: true` y `suspicious_reasons`
- Los hallazgos de consola con `detections` y `severity`

**15.1** Ejecuta una captura larga e intenta provocar errores SQL en DVWA. Comparte el archivo resultante con P5.

**15.2** Commit:

```
git add .
git commit -m "feat(devtools): integración con P2 y datos compartidos con P5"
git push origin feature/devtools
```

---

## BLOQUE 05 — Road Map Práctica 2

---

### Paso 16 — Elaborar el road map

El road map vale el **10% de la nota**. Créalo en `/docs/roadmap_practica2.md`:

```
# HookSuite – Road Map Práctica 2
```

```
## Fase 1 – Mejoras de rendimiento (semanas 1-2)
```

```
### 1. Caché de análisis de IA
```

Reusar resultados de análisis de IA para paquetes con el mismo URL pattern y tipo de payload.

Estimación: 3 días | Recursos: P5 | Reducción llamadas API: 40-60%

```
### 2. Procesamiento en batch para IA
```

Agrupar paquetes en lotes de 5-10 y enviarlos en una sola llamada a Claude.

Estimación: 2 días | Recursos: P5

```
---
```

```
## Fase 2 – Nuevas funcionalidades (semanas 3-4)
```

```
### 3. Interceptación de WebSockets
```

Capturar mensajes WebSocket con `Network.webSocketFrameReceived` del CDP.

Nuevo panel en frontend para visualizar tráfico WS.

Estimación: 5 días | Recursos: P4, P1, P5

```
### 4. Análisis de JavaScript estático con AST
```

Parsear el código JS de las páginas con `esprima/acorn` para detectar URLs hardcodeadas,

API keys expuestas y endpoints ocultos.

Estimación: 4 días | Recursos: P4, P5, P1

### ### 5. Detección de endpoints ocultos en JavaScript

Extraer automáticamente todas las URLs hardcodeadas en el código JavaScript y explorarlas con el spider de P3.

Estimación: 3 días | Recursos: P4, P3

---

## ## Fase 3 – Integraciones externas

### ### 6. Integración con base de datos de CVEs (NVD)

Cruzar tecnologías detectadas con CVEs conocidos via API pública de NVD.

Estimación: 3 días | Recursos: P4, P1

### ### 7. Exportación de informes en formatos estándar

Exportar hallazgos en JSON, XML y PDF auto-generado con IA.

Estimación: 4 días | Recursos: P5, P1

---

## ## Mejoras de seguridad de la herramienta

### ### 8. Autenticación de usuarios con JWT

Sistema de login para que solo usuarios autorizados accedan a HookSuite.

Estimación: 2 días | Recursos: P2, P1

### ### 9. Cifrado de comunicaciones internas

TLS en todas las comunicaciones entre módulos del sistema.

Estimación: 1 día | Recursos: P2

### ### 10. Rate limiting anti-abuso en el Intruder

Limitar peticiones por usuario y por minuto para evitar uso no autorizado.

Estimación: 1 día | Recursos: P2

---

## ## Tabla resumen

| #  | Mejora               | Fase      | Días | Responsable |
|----|----------------------|-----------|------|-------------|
| 1  | Caché análisis IA    | 1         | 3    | P5          |
| 2  | Batch para IA        | 1         | 2    | P5          |
| 3  | WebSockets           | 2         | 5    | P4, P1, P5  |
| 4  | JS estático AST      | 2         | 4    | P4, P5, P1  |
| 5  | Endpoints ocultos JS | 2         | 3    | P4, P3      |
| 6  | Integración CVEs     | 3         | 3    | P4, P1      |
| 7  | Exportación estándar | 3         | 4    | P5, P1      |
| 8  | Autenticación JWT    | Seguridad | 2    | P2, P1      |
| 9  | Cifrado interno      | Seguridad | 1    | P2          |
| 10 | Rate limiting        | Seguridad | 1    | P2          |

**\*\*Total estimado Práctica 2:\*\*** 28 días distribuidos entre los 5 roles

## 16.2 Commit:

```
git add .  
git commit -m "docs: road map Práctica 2 con 10 mejoras y estimaciones"  
git push origin feature/devtools
```

---

## BLOQUE 06 — Documentación y entrega

---

### Paso 17 — Abrir el Pull Request

#### 17.1 Commit final:

```
git add .  
git commit -m "feat(devtools): módulo Chrome DevTools completo HookSuite"  
git push origin feature/devtools
```

**17.2** Abre un Pull Request en GitHub de `feature/devtools` a `main`. Título: `feat(devtools): módulo Chrome DevTools completo HookSuite`. Incluye en la descripción una captura de la terminal mostrando paquetes capturándose y un paquete marcado como sospechoso.

📌 **CONTACTO CON P5 GitHub** — Notifica a P5 que el PR está listo para revisión.

---

### Paso 18 — Capturas para el informe PDF

Guarda en `/docs/capturas/devtools/`:

- `chrome_debugging.png` — Terminal mostrando Chrome lanzado con `--remote-debugging-port=9222`
- `cdp_conectado.png` — Salida de `verify_cdp_connection()` confirmando la conexión
- `paquetes_tiempo_real.png` — Terminal capturando paquetes en tiempo real
- `paquete_sospechoso.png` — Paquete marcado con `suspicious: true` y sus razones
- `hallazgo_consola.png` — Hallazgo sensible detectado en logs de consola
- `json_paquete.png` — JSON completo de un paquete capturado
- `roadmap_tabla.png` — La tabla del road map renderizada

---

### Paso 19 — Documentación técnica

Escribe `/docs/manual_devtools.md`:

```
# Documentación técnica – HookSuite Chrome DevTools  
  
## Stack tecnológico  
- Python 3.11
```

- Chrome DevTools Protocol (CDP) via WebSocket
- websockets 12.0 | httpx 0.27.0

## ## Qué es el CDP

API interna de Chrome activada con `--remote-debugging-port=9222`.  
Nuestro módulo Python se conecta via WebSocket y accede a datos de red, consola y DOM en tiempo real.

## ## Módulos

### CDPClient – Conexión WebSocket al CDP

### PacketBuilder – Transforma eventos CDP en objetos paquete unificados

### NetworkAnalyzer – Captura tráfico y detecta patrones sospechosos

### ConsoleAnalyzer – Detecta información sensible en logs de consola

## ## Flujo de datos

Chrome → CDP (eventos Network.\*) → PacketBuilder → NetworkAnalyzer  
→ PacketReporter → P2 Backend → P1 Frontend / P5 IA

## ## Decisiones de diseño

- CDP nativo en vez de librería de alto nivel: control total sobre eventos
- Filtrado de tráfico irrelevante: reduce ruido en ~70%
- Buffer local: los paquetes se guardan en disco si el backend no está disponible

## ## Limitaciones conocidas

- Solo captura tráfico del Chrome controlado, no otras aplicaciones
- Algunas respuestas pueden tener body vacío si Chrome libera la memoria antes
- Requiere Chrome instalado en la misma máquina que el módulo

## Paso 20 — Checklist final antes de la entrega

- ☐ Chrome se lanza con `--remote-debugging-port=9222` sin errores
- ☐ El cliente CDP conecta y recibe eventos de red
- ☐ El NetworkAnalyzer captura peticiones GET y POST
- ☐ Las peticiones POST incluyen el body correctamente
- ☐ El filtrado elimina imágenes, CSS y CDNs irrelevantes
- ☐ Los paquetes sospechosos se marcan con `suspicious: true`
- ☐ Los headers de seguridad ausentes se detectan
- ☐ Las cookies de sesión inseguras se detectan
- ☐ El ConsoleAnalyzer captura errores de la consola del navegador
- ☐ Los patrones sensibles en la consola se detectan
- ☐ El PacketReporter envía paquetes a P2 cuando está disponible
- ☐ El archivo `results/captured_packets.json` se genera correctamente
- ☐ Los datos de prueba se compartieron con P5
- ☐ El road map tiene 10 mejoras con estimaciones en `/docs/roadmap_practica2.md`
- ☐ Todos los commits siguen la convención del proyecto
- ☐ El PR a main está aprobado por P5
- ☐ Las capturas están en `/docs/capturas/devtools/`
- ☐ La documentación técnica está en `/docs/manual_devtools.md`

- ☐ `results/capture_output.txt` muestra una captura real funcionando
- 

*Manual técnico P4 Chrome DevTools — Proyecto HookSuite — Módulo de Ciberseguridad Avanzada 2026*